

模型能力

工具调用

 Copy page

💡 工具调用（Function Calling）允许 AI 模型调用外部函数和 API，极大扩展了智能体的能力边界，使其能够执行具体操作和获取实时数据。

功能特性

函数调用功能为 AI 模型提供了与外部系统交互的能力，支持多种复杂的应用场景和集成需求。

核心参数说明

- `tools` : 定义可调用的函数列表，包含函数名、描述和参数规范
- `tool_choice` : 控制函数调用策略，默认且仅支持 `auto`
- `model` : 使用支持函数调用的模型，如 `glm-5`、`glm-4.6`、`glm-4.7` 等

响应参数说明

函数调用响应中的关键字段：

- `tool_calls` : 包含模型决定调用的函数信息
- `function.name` : 被调用的函数名称
- `function.arguments` : 函数调用参数（JSON 格式字符串）
- `id` : 工具调用的唯一标识符

通过定义函数工具和处理函数调用，可以让 AI 模型执行各种外部操作：

>

Python SDK

安装 SDK

```
# 安装最新版本
pip install zai-sdk

# 或指定版本
pip install zai-sdk==0.2.2
```

验证安装

```
import zai
print(zai.__version__)
```

完整示例

```
from zai import ZhipuAiClient
```

```
# 初始化客户端
```

```
client = ZhipuAiClient(api_key='您的apikey')
```

```
# 定义天气查询函数
```

```
def get_weather(city: str) -> dict:
```

```
    """获取指定城市的天气信息"""
```

```
    # 这里应该调用真实的天气 API
```

```
    weather_data = {
```

```
        "city": city,
```

```
        "temperature": "22°C",
```

```
        "condition": "晴天",
```

```
        "humidity": "65%",
```

```
        "wind_speed": "5 km/h"
```

```
    }
```

```
    return weather_data
```

```
# 定义函数工具
```

```
tools = [
```

```
    {
```

```
        "type": "function",
```

```
        "function": {
```

```
            "name": "get_weather",
```

```
            "description": "获取指定城市的当前天气信息",
```

```
            "parameters": {
```

```
                "type": "object",
```

```
                "properties": {
```

```
                    "city": {
```

```
                        "type": "string",
```

```
                        "description": "城市名称, 例如: 北京、上海"
```

```
                    }
```

```
                },
```

```
                "required": ["city"]
```

```
            }
```

```
        }
```

```
    }
```

```
]
```

发起对话请求

```
response = client.chat.completions.create(
    model="glm-4.7", # 使用支持函数调用的模型
    messages=[
        {"role": "user", "content": "北京今天天气怎么样? "}
    ],
    tools=tools, # 传入函数工具
    tool_choice="auto" # 自动选择是否调用函数
)
```

处理函数调用

```
message = response.choices[0].message
messages = [{"role": "user", "content": "北京今天天气怎么样? "}
messages.append(message.model_dump())
```

```
if message.tool_calls:
    for tool_call in message.tool_calls:
        if tool_call.function.name == "get_weather":
            # 解析参数并调用函数
            args = json.loads(tool_call.function.arguments)
            weather_result = get_weather(args.get("city"))

            # 将函数结果返回给模型
            messages.append({
                "role": "tool",
                "content": json.dumps(weather_result, ensure_ascii=False),
                "tool_call_id": tool_call.id
            })
```

获取最终回答

```
final_response = client.chat.completions.create(
    model="glm-4.7",
    messages=messages,
    tools=tools
)
```

```
print(final_response.choices[0].message.content)
```

else:

```
print(message.content)
```

⚠️ 在使用函数调用时，请确保对外部 API 和数据库操作进行适当的安全验证和权限控制。

▸ 多功能助手

▸ 数据库查询

▸ 文件操作

▸ API集成

实践建议



函数设计原则

- 单一职责：每个函数只做一件事
- 清晰命名：函数名和参数名要有意义
- 完整描述：提供详细的函数和参数描述



安全考虑

- 输入验证：严格验证所有输入参数
- 权限控制：限制函数的访问权限
- 日志记录：记录函数调用日志

参数设计

```
{
  "type": "object",
  "properties": {
    "city": {
      "type": "string",
      "description": "城市名称, 支持中英文, 如: 北京、Beijing",
      "examples": ["北京", "上海", "New York"]
    },
    "unit": {
      "type": "string",
      "enum": ["celsius", "fahrenheit"],
      "description": "温度单位",
      "default": "celsius"
    }
  },
  "required": ["city"]
}
```

错误处理



def request_function(param: str) -> dict:

"""健壮的函数实现"""

try:

参数验证

if not param or not isinstance(param, str):

return {

"success": False,

"error": "参数无效",

"error_code": "INVALID_PARAM"

}

业务逻辑

result = process_data(param)

return {

"success": True,

"data": result,

"timestamp": datetime.now().isoformat()

}

except ValueError as e:

return {

"success": False,

"error": f"数据错误: {str(e)}",

"error_code": "DATA_ERROR"

}

except Exception as e:

return {

"success": False,

"error": f"系统错误: {str(e)}",

"error_code": "SYSTEM_ERROR"

}

输入验证



def secure_function(user_input: str) -> dict:



"""安全的函数实现"""

输入长度限制

```
if len(user_input) > 1000:
    return {"error": "输入过长"}
```

危险字符过滤

dangerous_chars = ['<', '>', '&', "'", '"']

```
if any(char in user_input for char in dangerous_chars):
    return {"error": "输入包含危险字符"}
```

SQL 注入防护

sql_keywords = ['DROP', 'DELETE', 'UPDATE', 'INSERT']

```
if any(keyword in user_input.upper() for keyword in sql_keywords):
    return {"error": "输入包含危险关键词"}
```

```
return {"success": True, "processed_input": user_input}
```

权限控制

```
def check_permissions(user_id: str, operation: str) -> bool:
```

"""检查用户权限"""

user_permissions = get_user_permissions(user_id)

```
return operation in user_permissions
```

```
def protected_function(user_id: str, operation: str, data: dict) -> dict:
```

"""需要权限验证的函数"""

```
if not check_permissions(user_id, operation):
```

```
    return {
        "success": False,
        "error": "权限不足",
        "error_code": "PERMISSION_DENIED"
    }
```

执行操作

```
return perform_operation(operation, data)
```

⚠️ 函数调用涉及代码执行，请确保实现适当的安全措施，包括输入验证、权限控制和错误处理。